

Davis–Putnam–Logemann–Loveland (DPLL) procedure for satisfiability

Luciano Serafini – Maurizio Lenzerini

FBK-IRST, Trento, Italy – Sapienza Università di Roma

Revised and adapted by F. Patrizi

Decision procedures - automated reasoning

Automated reasoning is the discipline that aims at solving basic decision problems in logic, the main of which are:

Four types of decision problems

- **Model Checking**($\mathcal{I}, \phi$): $\mathcal{I} \stackrel{?}{\models} \phi$ Does the interpretation $\mathcal{I}$ satisfy ϕ , or equivalently, is $\mathcal{I}$ a model of ϕ ?
- **Satisfiability**(ϕ): $\stackrel{?}{\exists} \mathcal{I} . \mathcal{I} \models \phi$ Is there a model of ϕ ?
- **Validity**(ϕ): $\stackrel{?}{\models} \phi$ Is every interpretation for ϕ a model of ϕ ?
- **Logical implication**(Γ, ϕ): $\Gamma \stackrel{?}{\models} \phi$ Is every model of the set of formulas Γ a model of ϕ as well?
- **Logical inference**(Γ, ϕ): $\Gamma \stackrel{?}{\vdash}_{\Sigma} \phi$ Is there a proof of ϕ in Σ from Γ ?

Model Checking

Model checking decision procedure

A model checking decision procedure, MCDP is an algorithm that checks if a formula ϕ is satisfied by an interpretation $\mathcal{I}$. Namely

$$\text{MCDP}(\phi, \mathcal{I}) = \text{true} \quad \text{if and only if} \quad \mathcal{I} \models \phi$$

$$\text{MCDP}(\phi, \mathcal{I}) = \text{false} \quad \text{if and only if} \quad \mathcal{I} \not\models \phi$$

Observation

The procedure of model checking returns for all inputs either **true** or **false** since for every interpretation $\mathcal{I}$ and for every formula ϕ , we have that either $\mathcal{I} \models \phi$ or $\mathcal{I} \not\models \phi$.

Observation

We have already seen a naive algorithm for model checking in propositional logic.

Satisfiability

Satisfiability decision procedure

A satisfiability decision procedure SDP is an algorithm that takes in input a formula ϕ and checks if ϕ is satisfiable. Namely

$$\begin{aligned}\text{SDP}(\phi) &= \textit{Satisfiable} && \text{if and only if } \mathcal{I} \models \phi \text{ for some } \mathcal{I} \\ \text{SDP}(\phi) &= \textit{Unsatisfiable} && \text{if and only if } \mathcal{I} \not\models \phi \text{ for all } \mathcal{I}\end{aligned}$$

When $\text{SDP}(\phi) = \textit{satisfiable}$, SDP can in addition return a model $\mathcal{I}$ (or, one of the models) of the formula ϕ .

Validity

Validity decision procedure

A decision procedure for Validity VDC, is an algorithm that checks whether a formula is valid. VDP can be based on a satisfiability decision procedure by exploiting the equivalence

ϕ is valid if and only if $\neg\phi$ is not satisfiable

$VDP(\phi) = true$ if and only if $SDP(\neg\phi) = Unsatisfiable$

$VDP(\phi) = false$ if and only if $SDP(\neg\phi) = Satisfiable$

When $SDP(\neg\phi)$ returns *Satisfiable*, it may in addition returns an interpretation $\mathcal{I}$, and $\mathcal{I}$ is called a **counter-model** for ϕ , in the sense that is an interpretation that falsifies ϕ .

Logical implication

Logical implication decision procedure

A decision procedure for logical consequence (or implication) LCDP is an algorithm that checks whether a formula ϕ is a logical consequence of a finite set of formulas $\Gamma = \{\gamma_1, \dots, \gamma_n\}$. LCDP can be implemented on the basis of satisfiability decision procedure by exploiting the property

$\Gamma \models \phi$ if and only if $\Gamma \cup \{\neg\phi\}$ is unsatisfiable

$LCDP(\Gamma, \phi) = \text{true}$ if and only if $SDP(\gamma_1 \wedge \dots \wedge \gamma_n \wedge \neg\phi) = \text{Unsatisfiable}$

$LCDP(\Gamma, \phi) = \text{false}$ if and only if $SDP(\gamma_1 \wedge \dots \wedge \gamma_n \wedge \neg\phi) = \text{Satisfiable}$

When $SDP(\gamma_1 \wedge \dots \wedge \gamma_n \wedge \neg\phi)$ returns *Satisfiable*, it may in addition returns an interpretation $\mathcal{I}$, and $\mathcal{I}$ is a **model for Γ** and a **counter-model for ϕ** , i.e., a model of Γ that falsifies ϕ .

Proof of the previous property

Theorem

$\Gamma \models \phi$ if and only if $\Gamma \cup \{\neg\phi\}$ is unsatisfiable.

Proof.

- $\Rightarrow$ If $\Gamma \models \phi$, then every model of Γ satisfies ϕ , and therefore we cannot find a model of Γ where $\neg\phi$ is true. It follows that $\Gamma \cup \{\neg\phi\}$ is unsatisfiable.
- $\Leftarrow$ If $\Gamma \cup \{\neg\phi\}$ is unsatisfiable, then either (a) Γ itself is unsatisfiable, or (b) Γ has at least one model, but $\neg\phi$ is false in every model of Γ . If (a) is the case, then Γ logically implies everything, and therefore $\Gamma \models \phi$. If (b) is the case, the set of models of Γ is nonempty, and ϕ is true in all such model. So, we have shown that $\Gamma \models \phi$ holds in all cases.



Davis–Putnam–Logemann–Loveland (DPLL) procedure for satisfiability

DPLL SAT procedure:

- Introduced in 1961 by Martin Davis, George Logemann and Donald W. Loveland;
- Refinement of the earlier Davis–Putnam procedure, which is a resolution-based procedure developed by Davis and Hilary Putnam in 1960;
- After more than 60 years the DPLL procedure still forms the basis for most efficient complete SAT solvers.

Original references:

- Martin Davis, Hilary Putnam: A Computing Procedure for Quantification Theory. J. ACM 7(3): 201-215 (1960)
- Martin Davis, George Logemann, Donald W. Loveland: A machine program for theorem-proving. Commun. ACM 5(7): 394-397 (1962)

Conjunctive Normal form

Definition

- A **literal** is either a propositional variable or the negation of a propositional variable.

$$p, \neg q$$

- A **clause** is a disjunction of literals.

$$(a \vee \neg b \vee c)$$

- A formula is in **conjunctive normal form**, if it is a conjunction of clauses.

$$(p \vee \neg q \vee r) \wedge (q \vee r) \wedge (\neg p \vee \neg q) \wedge r$$

Conjunctive Normal form

Conjunctive Normal form

A formula in conjunctive normal form has the following shape:

$$(l_{11} \vee \cdots \vee l_{1n_1}) \wedge \cdots \wedge (l_{m1} \vee \cdots \vee l_{mn_m})$$

equivalently written as

$$\bigwedge_{i=1}^m \left(\bigvee_{j=1}^{n_j} l_{ij} \right)$$

where l_{ij} is the j -th literal of the i -th clause composing ϕ

Example

$$\begin{array}{ll} (p \vee \neg q) \wedge (r \vee p \vee \neg r) \wedge (p \vee p) & p \vee q \\ p \wedge q, & p \wedge \neg q \wedge (r \vee s) \end{array}$$

Equi-satisfiable formulas

Two formulas ϕ and ϕ' are **equi-satisfiable** iff:

ϕ is satisfiable if and only if ϕ' is satisfiable

- If two formulas are equi-satisfiable, are they equivalent? **No!**
- **Example:** Any satisfiable formula (e.g., p) is equi-satisfiable with $\top$, but clearly, $p \equiv \top$ is not valid!
- **Another example:** Introducing names leads to equi-satisfiable formulas. E.g. the formula $a \wedge b$ is equi-satisfiable of the formula $(n \equiv a \wedge b) \wedge n$, but it is not true that

$$(a \wedge b) \equiv (n \wedge (a \wedge b \equiv n))$$

- Equi-satisfiability is a weaker notion than equivalence. But useful if all we want to do is determine satisfiability.

Tseitin's transformation

Tseitin's transformation ...

... converts any propositional formula ϕ into an equi-satisfiable formula ϕ' in CNF with only a linear increase in size.

Key ideas:

- give a “name” to every subformula (except for literals), and use this name as a fresh propositional letter “representing” the subformula
- for every formula ψ of the form $p_1 \equiv (p_2 \circ p_3)$ (where $\circ$ is a connective), there is a CNF formula $CNF(\psi)$ that is equivalent to ψ
- transform the original formula ψ into a conjunction of many $CNF(\psi_i)$ obtained from ψ , one for each subformula of ψ

* Gregory S. Tseitin - *On the complexity of proof in propositional calculus*, Leningrad, 1968.

Tseitin's transformation

If ψ is any formula, then for every subformula ϕ of ψ , we define its name n_ϕ as follows:

- n_ϕ is simply ϕ if ϕ is a literal;
- n_ϕ is a new propositional letter if ϕ is not a literal.

The Tseitin's transformation $T(\psi)$ of ψ is the conjunction of

- n_ψ
- $CNF(q \equiv \neg n_\phi)$ for every non-literal subformula of the form $\neg\phi$ having name q
- $CNF(q \equiv (n_{\phi_1} \circ n_{\phi_2}))$ for every subformula of the form $\phi_1 \circ \phi_2$ having name q , where $\circ \in \{\vee, \wedge, \rightarrow, \equiv\}$.

Tseitin's transformation

We still need to understand what **CNF** does. We proceed to analyze all cases.

- **CNF** $(q \equiv \neg p) = (\neg q \vee \neg p) \wedge (p \vee q)$
- **CNF** $(q \equiv (p \wedge r)) = (q \vee \neg p \vee \neg r) \wedge (\neg q \vee p) \wedge (\neg q \vee r)$
- **CNF** $(q \equiv (p \vee r)) = (\neg q \vee p \vee r) \wedge (q \vee \neg p) \wedge (q \vee \neg r)$
- **CNF** $(q \equiv (p \rightarrow r)) = (\neg q \vee \neg p \vee r) \wedge (p \vee q) \wedge (\neg r \vee q)$
- **CNF** $(q \equiv (p \equiv r)) = (q \vee p \vee r) \wedge (q \vee \neg p \vee \neg r) \wedge (\neg q \vee p \vee \neg r) \wedge (\neg q \vee \neg p \vee r)$

Tseitin's transformation

Theorem

For every formula ψ , $T(\psi)$ is satisfiable if and only if ψ is. Moreover, $T(\psi)$ is 3-CNF formula whose size is linear with respect to the size of ψ .

Tseitin's transformation - Example

Let ψ be the formula $(p \vee q) \rightarrow (p \wedge \neg r)$

- 1 For each non-literal subformula, introduce new letters:

x_1 for ψ , x_2 for $p \vee q$, x_3 for $p \wedge \neg r$

- 2 Stipulate equivalences and convert them to CNF:

$$\text{CNF}(x_1 \equiv (x_2 \rightarrow x_3)) \Rightarrow \phi_1 : (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (x_2 \vee x_1) \wedge (\neg x_3 \vee x_1)$$

$$\text{CNF}(x_2 \equiv (p \vee q)) \Rightarrow \phi_2 : (\neg x_2 \vee p \vee q) \wedge (\neg p \vee x_2) \wedge (\neg q \vee x_2)$$

$$\text{CNF}(x_3 \equiv (p \wedge \neg r)) \Rightarrow \phi_3 : (\neg x_3 \vee p) \wedge (\neg x_3 \vee \neg r) \wedge (\neg p \vee r \vee x_3)$$

- 3 The formula $T(\psi)$ is:

$$x_1 \wedge \phi_1 \wedge \phi_2 \wedge \phi_3$$

Properties of $\wedge$ and $\vee$

Commutativity of $\wedge$: $\phi \wedge \psi \equiv \psi \wedge \phi$

Commutativity of $\vee$: $\phi \vee \psi \equiv \psi \vee \phi$

Absorption of $\wedge$: $\phi \wedge \phi \equiv \phi$

Absorption of $\vee$: $\phi \vee \phi \equiv \phi$

Properties of clauses

Order of literals does not matter

If a clause C is obtained by **reordering the literals** of a clause C' then C and C' are equivalent.

E.g., $(p \vee q \vee r \vee \neg r) \equiv (\neg r \vee q \vee p \vee r)$

Multiple literals can be merged

If a clause contains **more than one occurrence of the same literal**, then it is equivalent to the clause obtained by deleting all but one such occurrences. E.g., $(p \vee q \vee r \vee q \vee \neg r) \equiv (p \vee q \vee r \vee \neg r)$

Clauses as set of literals

It follows from these properties that we can represent a **clause** as a **set of literals**, by leaving disjunction implicit and by ignoring replication and order of literals

$(p \vee q \vee r \vee \neg r)$ is represented by the set $\{p, q, r, \neg r\}$

Properties of formulas in CNF

Order of clauses does not matter

If a CNF formula ϕ is obtained by **reordering the clauses** of a CNF formula ϕ' then ϕ and ϕ' are equivalent. E.g.,

$$(a \vee b) \wedge (c \vee \neg b) \wedge (\neg b) \equiv (c \vee \neg b) \wedge (\neg b) \wedge (a \vee b)$$

Multiple clauses can be merged

If a CNF formula contains **more than one occurrence of the same clause**, then it is equivalent to the formula obtained by deleting all but one such occurrences. E.g.,

$$(a \vee b) \wedge (c \vee \neg b) \wedge (a \vee b) \equiv (a \vee b) \wedge (c \vee \neg b)$$

A CNF formula can be seen as a set of clauses

It follows from the properties of clauses and CNF formulas that we can represent a **CNF formula** as a **set of sets of literals**. E.g., $(\neg b) \wedge (b \vee a \vee b) \wedge (c \vee \neg b) \wedge (\neg b)$ is represented by $\{\{a, b\}, \{c, \neg b\}, \{\neg b\}\}$

Satisfiability of a set of clauses

- Let $\psi = \{C_1, \dots, C_n\}$
 - ▶ $\mathcal{I} \models \psi$ if and only if $\mathcal{I} \models C_i$ **for all** $i = 1..n$;
 - ▶ $\mathcal{I} \models C_i$ if and only if **for some** $l \in C_i$, $\mathcal{I} \models l$
- To check if an interpretation $\mathcal{I}$ satisfies ψ we do not necessarily need to know the truth values that $\mathcal{I}$ assigns to all the literals appearing in ψ .
- For instance, if $\mathcal{I}(p) = \text{true}$ and $\mathcal{I}(q) = \text{false}$, we can say that $\mathcal{I} \models \{\{p, q, \neg r\}, \{\neg q, s, q\}\}$, without considering the evaluations of $\mathcal{I}(r)$ and $\mathcal{I}(s)$.

Partial interpretation

A **partial interpretation** is a partial function that associates to **some propositional variables** of the alphabet P a truth value (either true or false) and can be undefined for the others.

Partial interpretations

- Partial interpretations can be used to construct models for a set of clauses $\psi = \{C_1, \dots, C_n\}$ **incrementally**
- Under a partial interpretation $\mathcal{I}$, literals and clauses can be **true**, **false** or **undefined**; a clause C
 - ▶ is true under $\mathcal{I}$ if **at least one of its literals is true**;
 - ▶ is false (or “conflicting”) under $\mathcal{I}$ if **all its literals are false**
 - ▶ is undefined (or “unresolved”) under $\mathcal{I}$, otherwise.
- The algorithms that exploit partial interpretations usually start with an empty interpretation (where the truth values of all propositional letters are undefined) and try to extend it step by step to the various variables occurring in $\{C_1, \dots, C_n\}$.

Simplification

- If λ is a literal of the form p , then $\neg\lambda$ denotes $\neg p$.
- If λ is a literal of the form $\neg p$, then $\neg\lambda$ denotes p .

Simplification of a formula by a literal

For any CNF formula ϕ and a literal λ , $\phi|_{\lambda}$ stands for the formula obtained from ϕ by

- removing all clauses containing the literal λ , and
- removing the literals $\neg\lambda$ in all remaining clauses

Example

For instance,

$$\{\{p, q, \neg r\}, \{\neg p, \neg r\}\}|_{\neg p} = \{\{q, \neg r\}\}$$

Unit propagation

Unit clause

A **unit clause** is a clause containing a single literal.

If the CNF formula ϕ contains a unit clause $\{\lambda\}$, then to satisfy ϕ the literal λ must be evaluated to True. As a consequence ϕ can be simplified using the procedure UNITPROPAGATION, that is invoked on ϕ and a partial interpretation $\mathcal{I}$, and returns a CNF formula and a partial interpretation.

Unit propagation

```
UNITPROPAGATION( $\phi, \mathcal{I}$ )  
  while  $\phi$  contains a unit clause  $\{\lambda\}$   
    if  $\lambda = p$ , then  $\mathcal{I}(p) := true$ ;  
    if  $\lambda = \neg p$ , then  $\mathcal{I}(p) := false$   
     $\phi := \phi|_{\lambda}$ ;  
  end;  
  return ( $\phi, \mathcal{I}$ )
```

The DPLL procedure

Example

UNITPROPAGATION($\{\{p\}, \{\neg p, \neg q\}, \{\neg q, r\}\}, \emptyset$)

$\{\{p\}, \{\neg p, \neg q\}, \{\neg q, r\}\}$

$\{\{p\}, \{\neg p, \neg q\}, \{\neg q, r\}\}|_p = \{\{\neg q\}, \{\neg q, r\}\} \quad \mathcal{I}(p) = true$

$\{\{\neg q\}, \{\neg q, r\}\}$

$\{\{\neg q\}, \{\neg q, r\}\}|_{\neg q} = \{\} \quad \mathcal{I}(q) = false$

the procedure returns $(\{\}, \mathcal{I})$

The DPLL procedure

Remark

In some cases, unit propagation applied to ϕ is enough to decide satisfiability of ϕ , for example when it terminates with one of the following two results:

- $(\{\}, \mathcal{I})$ (as in the example above), in which case the initial formula is satisfiable, and a satisfying interpretation can be easily extracted from $\mathcal{I}$;
- $(\phi, \mathcal{I})$, where $\{\} \in \phi$, in which case the original formula is unsatisfiable.

There are cases where UNITPROPAGATION does terminate with any of the cases above, i.e., when there is no unit clauses to consider, the CNF is nonempty, and it does not contain empty clauses.

Example: $\{\{p, q\}, \{\neg q, r\}\}$

The DPLL procedure

The Davis-Putnam-Logemann-Loveland procedure

To check the CNF formula ϕ for satisfiability, the procedure is invoked by $\text{DPLL}(\phi, \emptyset)$.

```
DPLL( $\phi, \mathcal{I}'$ )  
  ( $\psi, \mathcal{I}$ ) := UNITPROPAGATION( $\phi, \mathcal{I}'$ );  
  if  $\psi$  contains {}  
  then return ({{}},  $\emptyset$ )  
  elseif  $\psi = \{\}$  then return ( $\{\}$ ,  $\mathcal{I}$ )  
  else select a literal  $\lambda \in C \in \psi$ ;  
    if DPLL( $\psi \cup \{\{\lambda\}\}, \mathcal{I}$ ) = ( $\{\}$ ,  $\mathcal{I}''$ )  
    then return ( $\{\}$ ,  $\mathcal{I}''$ )  
    else return DPLL( $\psi \cup \{\{\neg\lambda\}\}, \mathcal{I}$ )
```

- UNITPROPAGATION realizes the “unit propagation rule”
- The last “if then else” realizes the “splitting rule”

Properties of the DPLL procedure

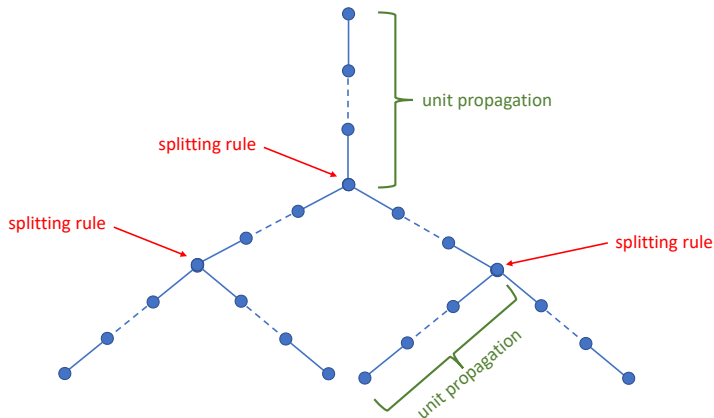
We first discuss the properties of `UNITPROPAGATION`. By observing that every execution of the instructions inside the `WHILE` loop eliminates one clause from the formula and possibly a set of literals from the other clauses of the input formula, we can easily conclude the following.

Theorem

For any $\phi, \mathcal{I}$, `UNITPROPAGATION`($\phi, \mathcal{I}$) terminates, and runs in polynomial time with respect to the size of ϕ .

Derivation tree

The computation done by $\text{DPLL}(\phi, \emptyset)$ can be described by a binary tree, in which every path from the root to a splitting node, or from a splitting node to another splitting node, or from a splitting node to a leaf is the sequence of operations of unit propagation.



Properties of the DPLL procedure

Since the length of every path from the root to a leaf is bound to n , where n is the number of variables in ϕ , we have that the size of the binary tree is at most 2^n . From this observation the following theorem on the worst-case time complexity follows.

Theorem

For any ϕ , $\text{DPLL}(\phi, \emptyset)$ terminates, and its time complexity is $O(2^m)$, where m is the size of ϕ .

Properties of the DPLL procedure

The following theorem sanctions the correctness of DPLL.

Theorem

For any ϕ , $\text{DPLL}(\phi, \emptyset)$ returns

- $(\{\}, \mathcal{I})$ if ϕ is satisfiable, and
- $(\{\{\}\}, \emptyset)$ if ϕ is unsatisfiable.

Note that when $\text{DPLL}(\phi, \emptyset)$ returns $(\{\}, \mathcal{I})$, $\mathcal{I}$ is a model of ϕ .

Horn clauses

Definition

A clause is a **Horn clause** if it has at most one positive literal. A **Horn formula** is a formula in CNF all of whose clauses are Horn clauses.

Theorem

A Horn formula ϕ is satisfiable if and only if $\text{UNITPROPAGATION}(\phi, \emptyset)$ returns $(\psi, \mathcal{I})$, with ψ different from $\{\{\}\}$.

Theorem

Satisfiability of Horn formulas can be solved in polynomial time.

We leave as an exercise the proof of the above theorems.